📚 جزوه سیستم عامل | فصل ۹: حافظه اصلی (Main Memory)

📝 بخش ۲: تخصیص همجوار حافظه (Contiguous Memory Allocation)

۱. مقدمه: تخصیص همجوار چیست؟

در روش‌های اولیه مدیریت حافظه، یک قانون سفت و سخت وجود داشت:

"هر پردازه (Process) باید در یک تکه حافظه پیوسته و پشت‌سرهم قرار بگیرد."

یعنی نمی‌توانستیم بخشی از برنامه را در آدرس ۱۰۰۰ و بخش دیگر را در آدرس ۵۰۰۰ بریزیم. همه باید کنار هم باشند. به این روش، **تخصیص همجوار** می‌گویند.

در این روش، حافظه اصلی معمولاً به دو بخش تقسیم می‌شود:

1. **سیستم‌عامل (OS):** که معمولاً در حافظه پایین (Low Memory) یا بالا (High Memory) مقیم است (همراه با بردار وقفه).
2. **پردازه‌های کاربر (User Processes):** بقیه حافظه که بین برنامه‌های ما تقسیم می‌شود.

۲. حفاظت حافظه (Memory Protection) 🛡️

چطور مطمئن شویم که پردازه A به حافظه پردازه B یا سیستم‌عامل تجاوز نمی‌کند؟ سیستم‌عامل از سخت‌افزار (MMU) کمک می‌گیرد و از دو ثبات استفاده می‌کند:

- **ثبات جابجایی (Relocation Register):** آدرس فیزیکی **شروع** پردازه را نگه می‌دارد. (مثلاً ۱۰۰۴۰).
- **ثبات حد (Limit Register): طول (اندازه)** پردازه را نگه می‌دارد. (مثلاً ۷۴۶۰۰).

نحوه عملکرد سخت‌افزار (نکته امتحانی):

1. CPU یک آدرس منطقی تولید می‌کند.

2. سخت‌افزار چک می‌کند: آیا آدرس منطقی < Limit است؟
   - اگر **خیر**: خطا (Trap) تولید می‌شود (دسترسی غیرمجاز).
   - اگر **بله**: آدرس فیزیکی = آدرس منطقی + Relocation.

این مکانیزم دو فایده دارد:

1. **حفاظت:** برنامه‌ها ایزوله می‌شوند.
2. **انعطاف:** سیستم‌عامل می‌تواند جای برنامه را در حافظه تغییر دهد، فقط با تغییر عدد داخل "ثبات جابجایی".

۳. تخصیص حافظه (Memory Allocation)

چطور فضای خالی را به برنامه‌ها بدهیم؟ یکی از روش‌های رایج، استفاده از **پارتیشن‌های متغیر (Variable Partitions)** است.

- در ابتدا، کل حافظه کاربر یک "حفره" (Hole) بزرگ و خالی است.
- وقتی برنامه‌ها می‌آیند، تکه‌ای از این حفره به آن‌ها داده می‌شود.
- وقتی برنامه‌ای تمام می‌شود، حافظه‌اش آزاد شده و دوباره تبدیل به حفره می‌شود.
- اگر دو حفره کنار هم آزاد شوند، با هم ادغام شده و یک حفره بزرگتر می‌سازند.

الگوریتم‌های انتخاب حفره (بسیار مهم)

وقتی یک برنامه جدید با اندازه n وارد می‌شود و ما چندین حفره خالی داریم، کدام را به او بدهیم؟

اولین گنجایش (First-Fit):
- لیست حفره‌ها را از اول می‌گردیم و **اولین حفره‌ای** که اندازه‌اش کافی باشد ($size$ \ge n$) را انتخاب می‌کنیم.
- **مزیت:** معمولاً **سریع‌ترین** روش است.

بهترین گنجایش (Best-Fit):
- کل لیست حفره‌ها را می‌گردیم و **کوچک‌ترین حفره‌ای** که اندازه‌اش کافی باشد را انتخاب می‌کنیم.
- *منطق:* حفره‌های بزرگ را نگه داریم برای برنامه‌های بزرگ بعدی.

- **عیب:** کند است (چون باید همه را بگردد) و باعث تولید حفره‌های بسیار ریز و بی‌استفاده می‌شود.

بدترین گنجایش (Worst-Fit):

- کل لیست را می‌گردیم و **بزرگ‌ترین حفره موجود** را انتخاب می‌کنیم.
- *منطق:* وقتی از بزرگترین حفره برداریم، تکه باقیمانده‌اش هنوز آنقدر بزرگ هست که به درد بخورد.
- **عیب:** حفره‌های بزرگ و ارزشمند را سریع مصرف و خراب می‌کند.

جمع‌بندی کنکوری: روش‌های **First-Fit** و **Best-Fit** از نظر بهره‌وری حافظه بهتر از Worst-Fit هستند. First-Fit سریع‌تر است.

۴. پارگی حافظه (Fragmentation) 💔

بزرگترین مشکل تخصیص همجوار، تکه‌تکه شدن حافظه است.

الف)پارگی‌خارجی(ExternalFragmentation)

- **تعریف:** وقتی فضای خالیِ **کل** حافظه برای اجرای یک برنامه کافی است، اما این فضا **پیوسته نیست** (تکه‌تکه است) و نمی‌توانیم آن را به برنامه بدهیم.
- **مثال:** ما ۱۰۰ مگابایت جای خالی داریم، اما به صورت ۱۰ تکه ۱۰ مگابایتی جدا از هم. اگر برنامه ۲۰ مگابایت بخواهد، نمی‌توانیم اجرا کنیم.
- **قانون ۵۰ درصد:** آمار نشان می‌دهد در روش First-Fit، حدود $2/N$ بلاک به خاطر پارگی هدر می‌رود (یعنی یک‌سوم حافظه غیرقابل استفاده می‌شود).

ب)پارگی‌داخلی(InternalFragmentation)

- **تعریف:** وقتی حافظه را در بلاک‌های ثابت (مثلاً ۱۰۰۰ بایتی) تقسیم می‌کنیم، اگر برنامه‌ای ۱۸۶۰ بایت بخواهد، مجبوریم دو بلاک (۲۰۰۰ بایت) به او بدهیم. آن ۱۴۰ بایتِ اضافه که داخل بلاک دوم خالی می‌ماند، پارگی داخلی است.
- *نکته:* این پارگی **درون** فضایی است که به پردازه داده شده است.

راه‌حل‌های‌پارگی‌خارجی:

فشرده‌سازی (Compaction):

- سیستم‌عامل برنامه‌ها را در حافظه جابجا می‌کند (هل می‌دهد به یک سمت) تا تمام حفره‌های خالی در سمت دیگر جمع شوند و یک حفره بزرگ بسازند.

- *شرط لازم:* آدرس‌دهی باید در **زمان اجرا (Dynamic)** باشد.
- *عیب:* بسیار زمان‌بر و پرهزینه است.

استفاده از روش‌های غیرهمجوار (مثل صفحه‌بندی):

- این روش (که در بخش بعدی درس است) اجازه می‌دهد برنامه تکه‌تکه شود و مشکل پارگی خارجی را کامل حل می‌کند.

📊 جدول مقایسه نهایی

ویژگی	تخصیص همجوار
محل قرارگیری	یک تکه پیوسته.
حفاظت	ثبات‌های Base و Limit.
مشکل اصلی	پارگی خارجی (External Fragmentation).
بهترین الگوریتم	First-Fit (سریع و خوب).
راه‌حل پارگی	فشرده‌سازی (Compaction).